



Based on this input, the web application displays “Hi Rakesh” at the top of the page.

An attacker might have a user visit a malicious version of the URL that looks like this: <https://example.com/profile?userscript>some> malicious code/script> if the user parameter is not verified to ensure that it only contains expected data.

The malicious script is included in the response, which is subsequently performed in the browser, most likely without the user’s knowledge. This is an example of a reflected cross-site scripting (XSS) attack. The malicious code is “reflected” back to the user who made the request right away.

Here are a few things to prevent XSS attacks:

- **Awareness and education** - Make sure that all developers, website designers, and QA teams are aware of the ways that hackers use to attack vulnerabilities, and that they have access to coding guidelines and best practices. This includes correct application-specific escaping and encoding strategies (JavaScript, HTML, etc.).
- **Clean up the input** - Never trust the validity of user input data, whether it’s for internal or public web pages. Any data fields should be screened and validated, especially if they will be used in HTML output.
- **Scan code** - Implement software that scans code for vulnerabilities, such as cross-site scripting, to verify that best practices are followed and that XSS and other OWASP-listed vulnerabilities are not exploited.
- **Content Security Policy** - Use Content Security Policy (CSP) to define what a website can do, lowering the chance of an XSS attack. XSS can be